

Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business

Go from Data to Strategy: Tepper School of Business

Save and Load Your PyTorch Models

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch** 💬 4[Share](#) [Share](#) [Post](#)

A deep learning model is a mathematical abstraction of data, in which a lot of parameters are involved. Training these parameters can take hours, days, and even weeks but afterward, you can make use of the result to apply on new data. This is called inference in machine learning. It is important to know how we can preserve the trained model in disk and later, load it for use in inference. In this post, you will discover how to save your PyTorch models to files and load them up again to make predictions. After reading this chapter, you will know:

- What are states and parameters in a PyTorch model
- How to save model states
- How to load model states

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Save and Load Your PyTorch Models
Photo by Joseph Chan. Some rights reserved.

Overview

This post is in three parts; they are

- Build an Example Model
- What's Inside a PyTorch Model
- Accessing `state_dict` of a Model

Build an Example Model

Let's start with a very simple model in PyTorch. It is a model based on the iris dataset. You will load the dataset using scikit-learn (which the targets are integer labels 0, 1, and 2) and train a neural network for this multiclass classification problem. In this model, you used log softmax as the output activation so you can combine with the negative log likelihood loss function. It is equivalent to no output activation combined with cross entropy loss function.

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import load_iris
5 from sklearn.model_selection import train_test_split
6
7 # Load data into NumPy arrays
8 data = load_iris()
9 X, y = data["data"], data["target"]
10
11 # convert NumPy array into PyTorch tensors
12 X = torch.tensor(X, dtype=torch.float32)
13 y = torch.tensor(y, dtype=torch.long)
14
15 # split
16 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
17
18 # PyTorch model
19 class Multiclass(nn.Module):
20     def __init__(self):
21         super().__init__()
22         self.hidden = nn.Linear(4, 8)
23         self.act = nn.ReLU()
24         self.output = nn.Linear(8, 3)
25         self.logsoftmax = nn.LogSoftmax(dim=1)
26
27     def forward(self, x):
28         x = self.act(self.hidden(x))
29         x = self.logsoftmax(self.output(x))
30         return x
31
32 model = Multiclass()
33
34 # loss metric and optimizer
35 loss_fn = nn.NLLLoss()
36 optimizer = optim.Adam(model.parameters(), lr=0.001)
37
38 # prepare model and training parameters
39 n_epochs = 100
40 batch_size = 5
41 batch_start = torch.arange(0, len(X), batch_size)
42
43 # training loop
44 for epoch in range(n_epochs):
45     for start in batch_start:
46         # take a batch
47         X_batch = X_train[start:start+batch_size]
48         y_batch = y_train[start:start+batch_size]
49         # forward pass
50         y_pred = model(X_batch)
51         loss = loss_fn(y_pred, y_batch)
52         # backward pass
53         optimizer.zero_grad()
54         loss.backward()
55         # update weights
56         optimizer.step()

```

With such a simple model and small dataset, it shouldn't take a long time to finish training. Afterwards, we can confirm that this model works, by evaluating it with the test set:

```

1 ...
2 y_pred = model(X_test)
3 acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
4 print("Accuracy: %.2f" % acc)

```

It prints, for example,

```

1 Accuracy: 0.96

```

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

What's Inside a PyTorch Model

PyTorch model is an object in Python. It holds some deep learning building blocks such as various kinds of layers and activation functions. It also knows how to connect them so it can produce you an output from your input tensors. The algorithm of a model is fixed at the time you created it, however, it has trainable parameters that is supposed to be modified during training loop so the model can be more accurate.

You saw how to get the model parameters when you set up the optimizer for your training loop, namely,

```
1 optimizer = optim.Adam(model.parameters(), lr=0.001)
```

The function `model.parameters()` give you a generator that reference to each layers' trainable parameters in turn in the form of PyTorch tensors. Therefore, it is possible for you to make a copy of them or overwrite them, for example:

```
1 # create a new model
2 newmodel = Multiclass()
3 # ask PyTorch to ignore autograd on update and overwrite parameters
4 with torch.no_grad():
5     for newtensor, oldtensor in zip(newmodel.parameters(), model.parameters()):
6         newtensor.copy_(oldtensor)
7 # test with new model using copied tensor
8 y_pred = newmodel(X_test)
9 acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
10 print("Accuracy: %.2f" % acc)
```

Which the result should be exactly the same as before since you essentially made the two models identical by copying the parameters.

However, this is not always the case. Some models has **non-trainable parameters**. One example is the batch normalization layer that is common in many convolution neural networks. What it does is to apply normalization on tensors that produced by its previous layer and pass on the normalized tensor to its next layer. It has two parameters: The mean and standard deviation, which are learned from your input data during training loop but not trainable by the optimizer. Therefore these are not part of `model.parameters()` but equally important.

Accessing state_dict of a Model

To access all parameters of a model, trainable or not, you can get it from `state_dict()` function. From the model above, this is what you can get:

```
1 import pprint
2 pp = pprint.PrettyPrinter(indent=4)
3 pp.pprint(model.state_dict())
```

The model above produces the following:

```
1 OrderedDict([ ( 'hidden.weight',
2                 tensor([[ 0.1480,  0.0336,  0.3425,  0.2832],
3                 [ 0.5265,  0.8587, -0.7023, -1.1149],
4                 [ 0.1620,  0.8440, -0.6189, -0.6513],
5                 [-0.1559,  0.0393, -0.4701,  0.0825],
6                 [ 0.6364, -0.6622,  1.1150,  0.9162],
7                 [ 0.2081, -0.0958, -0.2601, -0.3148],
8                 [-0.0804,  0.1027,  0.7363,  0.6068],
9                 [-0.4101, -0.3774, -0.1852,  0.1524]])),
10              ( 'hidden.bias',
11                tensor([ 0.2057,  0.7998, -0.0578,  0.1041, -0.3903, -0.4521, -0.5307, -0.1532])),
12              ( 'output.weight',
13                tensor([[ -0.0954,  0.8683,  1.0667,  0.2382, -0.4245, -0.0409, -0.2587, -0.0745],
14                [-0.0829,  0.8642, -1.6892, -0.0188,  0.0420, -0.1020,  0.0344, -0.1210],
15                [-0.0176, -1.2809, -0.3040,  0.1985,  0.2423,  0.3333,  0.4523, -0.1928]])),
16              ( 'output.bias', tensor([ 0.0998,  0.6360, -0.2990]))])
```

It is called `state_dict` because all state variables of a model are here. It is an `OrderedDict` object from Python's built-in `collections` module. All components from a PyTorch model has a name and so as the parameters therein. The `OrderedDict` object allows you to map the weights back to the parameters correctly by matching their names.

This is how you should save and load the model: Fetch the model states into an `OrderedDict`, serialize and save it to disk. For inference, you create a model first (without training), and load the states. In Python, the native format for serialization is pickle:

```
1 import pickle
2
3 # Save model
4 with open("iris-model.pickle", "wb") as fp:
5     pickle.dump(model.state_dict(), fp)
6
7 # Create new model and load states
8 newmodel = Multiclass()
9 with open("iris-model.pickle", "rb") as fp:
10     newmodel.load_state_dict(pickle.load(fp))
11
12 # test with new model using copied tensor
13 y_pred = newmodel(X_test)
```

```
14 acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
15 print("Accuracy: %.2f" % acc)
```

You know it works because the model you didn't train produced the same result as the one you trained.

Indeed, the recommended way is to use the PyTorch API to save and load the states, instead of using pickle manually:

```
1 # Save model
2 torch.save(model.state_dict(), "iris-model.pth")
3
4 # Create new model and load states
5 newmodel = Multiclass()
6 newmodel.load_state_dict(torch.load("iris-model.pth"))
7
8 # test with new model using copied tensor
9 y_pred = newmodel(X_test)
10 acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
11 print("Accuracy: %.2f" % acc)
```

The *.pth file is indeed a zip file of some pickle files created by PyTorch. It is recommended because PyTorch can store additional information in it. Note that you stored only the states but not the model. You still need to create the model using Python code and load the states into it. If you wish to store the model as well, you can pass in the entire model instead of the states:

```
1 # Save model
2 torch.save(model, "iris-model-full.pth")
3
4 # Load model
5 newmodel = torch.load("iris-model-full.pth")
6
7 # test with new model using copied tensor
8 y_pred = newmodel(X_test)
9 acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
10 print("Accuracy: %.2f" % acc)
```

But remember, due to the nature of Python language, doing so does not relieve you from keeping the code of the model. The newmodel object above is an instance of Multiclass class that you defined before. When you load the model from disk, Python need to know in detail how this class is defined. If you run a script with just the line torch.load(), you will see the following error message:

```
1 Traceback (most recent call last):
2   File "<stdin>", line 1, in <module>
3   File ".../torch/serialization.py", line 789, in load
4     return _load(opened_zipfile, map_location, pickle_module, **pickle_load_args)
5   File ".../torch/serialization.py", line 1131, in _load
6     result = unpickler.load()
7   File ".../torch/serialization.py", line 1124, in find_class
8     return super().find_class(mod_name, name)
9   AttributeError: Can't get attribute 'Multiclass' on <module '__main__' (built-in)>
```

That's why it is recommended to save only the state dict rather than the entire model.

Putting everything together, the following is the complete code to demonstrate how to create a model, train it, and save to disk:

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import load_iris
5 from sklearn.model_selection import train_test_split
6
7 # Load data into NumPy arrays
8 data = load_iris()
9 X, y = data["data"], data["target"]
10
11 # convert NumPy array into PyTorch tensors
12 X = torch.tensor(X, dtype=torch.float32)
13 y = torch.tensor(y, dtype=torch.long)
14
15 # split
16 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
17
18 # PyTorch model
19 class Multiclass(nn.Module):
20     def __init__(self):
21         super().__init__()
22         self.hidden = nn.Linear(4, 8)
23         self.act = nn.ReLU()
24         self.output = nn.Linear(8, 3)
25         self.logsoftmax = nn.LogSoftmax(dim=1)
26
27     def forward(self, x):
28         x = self.act(self.hidden(x))
29         x = self.logsoftmax(self.output(x))
30         return x
31
32 model = Multiclass()
33
34 # loss metric and optimizer
35 loss_fn = nn.NLLLoss()
36 optimizer = optim.Adam(model.parameters(), lr=0.001)
37
38 # prepare model and training parameters
```

```

39 n_epochs = 100
40 batch_size = 5
41 batch_start = torch.arange(0, len(X), batch_size)
42
43 # training loop
44 for epoch in range(n_epochs):
45     for start in batch_start:
46         # take a batch
47         X_batch = X_train[start:start+batch_size]
48         y_batch = y_train[start:start+batch_size]
49         # forward pass
50         y_pred = model(X_batch)
51         loss = loss_fn(y_pred, y_batch)
52         # backward pass
53         optimizer.zero_grad()
54         loss.backward()
55         # update weights
56         optimizer.step()
57
58 # Save model
59 torch.save(model.state_dict(), "iris-model.pth")

```

And the following is how to load the model from disk and run it for inference:

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.datasets import load_iris
5 from sklearn.model_selection import train_test_split
6
7 # Load data into NumPy arrays
8 data = load_iris()
9 X, y = data["data"], data["target"]
10
11 # convert NumPy array into PyTorch tensors
12 X = torch.tensor(X, dtype=torch.float32)
13 y = torch.tensor(y, dtype=torch.long)
14
15 # PyTorch model
16 class Multiclass(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.hidden = nn.Linear(4, 8)
20         self.act = nn.ReLU()
21         self.output = nn.Linear(8, 3)
22         self.logsoftmax = nn.LogSoftmax(dim=1)
23
24     def forward(self, x):
25         x = self.act(self.hidden(x))
26         x = self.logsoftmax(self.output(x))
27         return x
28
29 # Create new model and load states
30 model = Multiclass()
31 with open("iris-model.pickle", "rb") as fp:
32     model.load_state_dict(pickle.load(fp))
33
34 # Run model for inference
35 y_pred = model(X_test)
36 acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
37 print("Accuracy: %.2f" % acc)

```

Further Readings

This section provides more resources on the topic if you are looking to go deeper.

- [Saving and loading models from PyTorch tutorial](#)

Summary

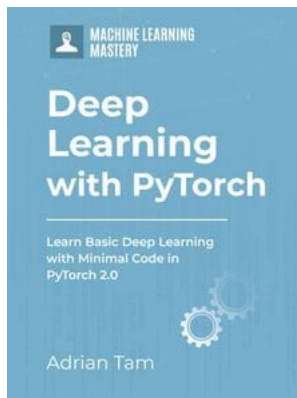
In this post, you learned how to keep a copy of your trained PyTorch model in disk and how to reuse it. In particular, you learned

- What are parameters and states in a PyTorch model
- How to save all necessary states from a model to disk
- How to rebuild a working model from the saved states

Get Started on Deep Learning with PyTorch!

Learn how to build deep learning models

...using the newly released PyTorch 2.0 library



Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

Share

Share

Post

More On This Topic



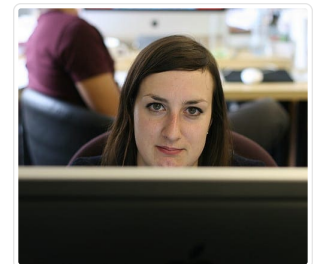
How to Save and Load Your Keras Deep Learning Model



Save and Load Machine Learning Models in Python with...



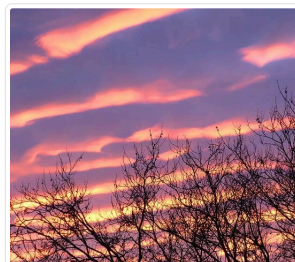
How to Load, Convert, and Save Images With the Keras API



How To Load Your Machine Learning Data Into R



Save And Finalize Your Machine Learning Model in R



How to Save Your Machine Learning Model and Make...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

< [How to Grid Search Hyperparameters for PyTorch Models](#)

[Using Activation Functions in Deep Learning Models](#) >

4 Responses to *Save and Load Your PyTorch Models*

**Javier** March 10, 2023 at 6:35 am #

REPLY ↩

Thank you it was really useful, just a comment. In the final example the loading operation is shown using pickle instead of torch.load. Torch load will be more consistent with the saving code showed before.

**James Carmichael** March 10, 2023 at 7:59 am #

REPLY ↩

You are very welcome Javier! Thank you for your feedback and suggestion!

**Poult** August 10, 2023 at 4:48 pm #

REPLY ↩

Hello, I think in the last windows of code, in the 31-32 line:
it could be: `with open("iris-model.pth", "rb") as fp:`
`model.load_state_dict(torch.load(fp))`

**James Carmichael** August 11, 2023 at 8:15 am #

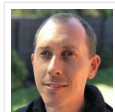
REPLY ↩

Thank you for your feedback and suggestion Poult!

Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT

**Welcome!**I'm *Jason Brownlee* PhDand I **help developers** get results with **machine learning**.[Read more](#)**Never miss a tutorial:**

Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The **Deep Learning with PyTorch** EBook
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. Visit our [corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved